

UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



GROUP PROJECT REPORT
Course: INTRODUCTION TO SOFTWARE ENGINEERING
Project Proposal

Data management and analysis system through an LLM - DataPilot

GROUP 10

Student ID	FULL NAME
22127408	Kha Vĩnh Thuận
24127075	Nguyễn Sơn Lộc
24127139	Lê Thành Trung

SUPERVISOR:
Ngô Ngọc Đăng Khoa

Ho Chi Minh City, June 2026

Contents

1	Member Contribution Assessment	2
2	Preliminary Problem Statement	3
3	Proposed Solution	4
3.1	Software	4
3.1.1	Features	4
3.1.2	Software Architecture	6
3.2	Hardware	7
4	Development Plan	7
4.1	Requirements analysis	7
4.2	Software design	8
4.3	Implementation	10
4.4	Testing	12
4.5	Deployment and maintenance	13
5	Human Resources & Costing Plan	13
6	Tools setup	14
7	AI Usage Declaration	15
7.1	Detailed declaration for each tool	15
7.2	Team commitments	18
8	Presentation	18
9	Reflective Report	18

1 Member Contribution Assessment

Member	Main role	Contribution (%)
24127075 – Nguyễn Sơn Lộc	Problem Statement, Architecture, Video	33
24127139 – Lê Thành Trung	Features, Hardware, HR & Costing	33
22127408 – Kha Vĩnh Thuận	Development Plan, AI Declaration	33

Task Evidence: tasks are managed on Jira, and each task is assigned to exactly one member. The screenshots below show Task Name, Assignee, Status, Assigned Date and Completion Date.

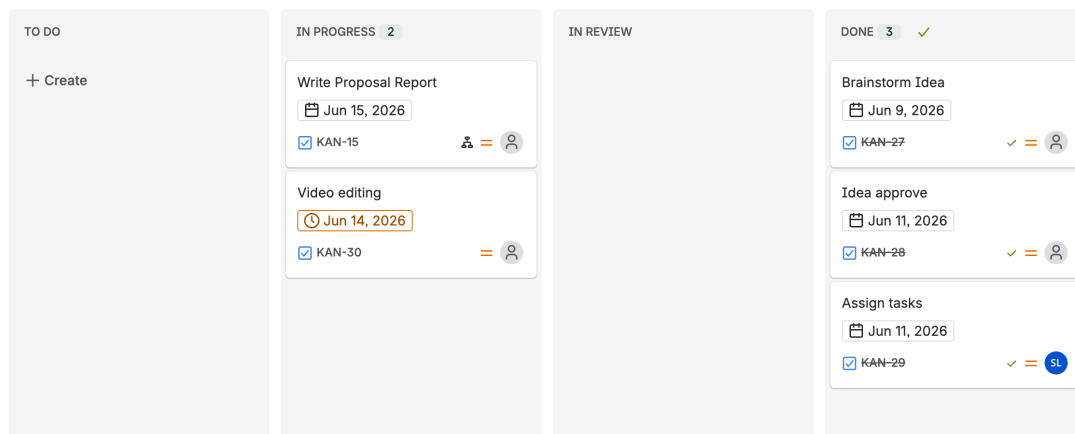


Figure 1: Jira board: Task Name, Assignee, Status, Assigned Date, Completion Date.

54% Done			
Summary	Status	Assignee	Comments
2. Problem Statement	DONE ✓	SL Sơn Lộc	1 comment
3.1 Features (user story map)	DONE ✓	TT Trung Lê Thành	1 comment
3.2 Architecture (LLM/MCP)	DONE ✓	SL Sơn Lộc	1 comment
3.3 Hardware	DONE ✓	TT Trung Lê Thành	1 comment
4. Development Plan	IN REVIEW	Vĩnh Thuận	1 comment
5. HR & Costing	DONE ✓	TT Trung Lê Thành	1 comment
6. Tools setup (repo, Jira, Slack)	DONE ✓	Unassigned	Add comment
7. AI Declaration	IN REVIEW	Vĩnh Thuận	1 comment
8. Video	IN PROGRESS	SL Sơn Lộc	1 comment
9. Reflective Report	IN PROGRESS	Unassigned	Add comment
Check All	IN PROGRESS	Unassigned	Add comment

Figure 2: Jira board: Task Name, Assignee, Status

2 Preliminary Problem Statement

Written by: 24127075 – Nguyễn Sơn Lộc

Edited by: 24127075 – Nguyễn Sơn Lộc

Reviewed by: 22127408 – Kha Vĩnh Thuận

Context and problem

Database administration today poses a significant technical barrier for non-expert users. Specifically, in order to view, search or edit data, a user is forced to:

- Be fluent in the SQL query language.
- Use specialized tools such as DBeaver, phpMyAdmin or pgAdmin.

In practice, many people who need to work with data regularly — sales staff, accountants, managers — do not have these skills. They have to depend on the technical team even for the simplest needs, for example:

- “How many orders did we have this month?”
- “Export the list of new customers to Excel.”

This wastes time for both sides and creates an operational bottleneck. Even for technical users, repetitive operations such as generating periodic reports, importing/exporting data and checking the schema still require a lot of manual effort and are error-prone.

Proposed solution (preliminary level)

The team is building a **full-featured database management system in which the LLM acts as the interface and the main orchestrating program** (*LLM as main program*). The key points of the solution are:

- Users interact using natural language instead of writing SQL by hand.
- The LLM calls *tools* provided by the system to perform tasks: CRUD records, read the schema, execute SQL (Text2SQL), import/export CSV, and generate periodic reports.
- A traditional Web UI (Dashboard, Database Explorer, Schema, Import/Export, Reports, Chat/Query) serves users who want to operate directly.
- Both the Web UI and the LLM call the same REST API layer — ensuring consistency and avoiding duplicated logic.
- All functionality is exposed through an **MCP server** (Model Context Protocol), allowing Claude Desktop / Claude Code to be used as an alternative interface without opening the Web UI.

Operating environment

- **Client:** a web browser supporting HTML5 (Chrome, Edge, Firefox); or Claude Desktop / Claude Code connecting via MCP.
- **Server:** a local machine running the Python backend (FastAPI) and the MCP server; no dedicated server or Docker required.
- **Database:** SQLite — lightweight, no separate server to install, suitable for a local environment. Default dataset: Northwind (customers, orders, products, employees).
- **LLM:** Google Gemini 2.0 Flash — accessed through the Gemini API (free tier). An Internet connection is required.

Design and implementation constraints

- **Programming languages:** Python 3.13 + FastAPI for the backend; React 18 + Vite for the frontend.
- **Communication:** client–server over REST API (JSON); the MCP server complies with the Model Context Protocol specification.
- **Data safety:** queries generated by the LLM must pass through a control layer (validate/sandbox) before execution; delete/update operations require a confirmation mechanism.
- **Project management:** documents follow the course template; source code is version-controlled on GitHub.

Note: this is a preliminary problem statement; the detailed statement will be presented in the analysis document (Template 1).

3 Proposed Solution

3.1 Software

3.1.1 Features

Written by: 24127139 – Lê Thành Trung

Edited by: 24127139 – Lê Thành Trung

Reviewed by: 22127408 – Kha Vĩnh Thuận

Example user stories for managing a company’s data:

Need	Requirement	Feature
As a director, I want to know the company's total revenue for the month, quarter and year	Overview statistics, intuitive charts	Dashboard
As an HR manager, I want to add a new employee, read employee information, raise/lower an employee's salary, and dismiss an employee	Add, read, update, delete and search employee data	Browse / CRUD records
As a database maintainer, I want an overview of how the data is stored and how the tables relate to one another so I can fix issues if any arise	Present the structure of the tables and the relationships between them	Show Schema
As an accountant, I want to consolidate the revenue of the branches into a single database, as well as export the company's revenue and capital into a financial report	Import / Export data	Import / Export CSV
As a product data analyst, I want to quickly list products in ascending order of profit without having to write the query myself	Natural-language Q&A, automatically querying data on request	Text2SQL / Chat AI
As a manager, I want to receive a consolidated report on revenue, order count and best-selling products over the past week without having to write the query myself	Automatically consolidate business figures on a periodic basis	Weekly Report (AI skill)

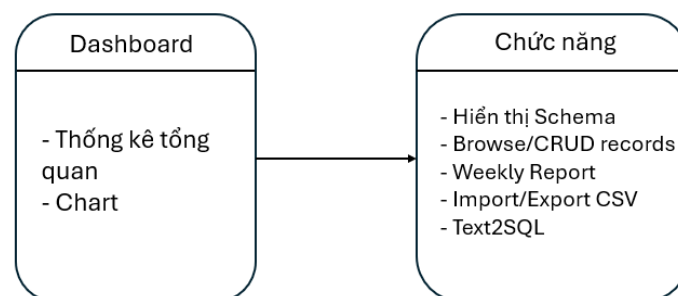


Figure 3: User Story Map

3.1.2 Software Architecture

Written by: 24127075 – Nguyễn Sơn Lộc

Edited by: 24127075 – Nguyễn Sơn Lộc

Reviewed by: 22127408 – Kha Vĩnh Thuận

The system architecture is designed around the principle of **LLM as main program**: the LLM is not just a chatbot that answers questions, but the central orchestrating component that accesses the entire functionality of the system through four mechanisms: **tools**, **API**, **MCP** and **skills**.

Main components

- **Web UI (React):** a traditional interface with the Dashboard, Database Explorer, Schema, Import/Export, Reports and Chat/Query pages.
- **REST API layer:** a single API layer that provides all database-management functionality. Both the Web UI and the LLM Orchestrator call this same layer — ensuring consistency and avoiding duplicated logic.
- **LLM Orchestrator:** receives natural-language requests, plans, and calls the *tools*: CRUD records, read the schema, execute SQL (Text2SQL), import/export CSV, and generate reports.
- **MCP Server:** exposes all tools and resources following the Model Context Protocol standard, allowing Claude Desktop / Claude Code to become an alternative interface to the web UI.
- **Skills:** the **weekly-report** skill automatically runs a chain of queries and consolidates a structured report on a periodic schedule.
- **Database:** SQLite stores user data; the default dataset is Northwind — a sample commercial database containing customers, orders, products and employees.

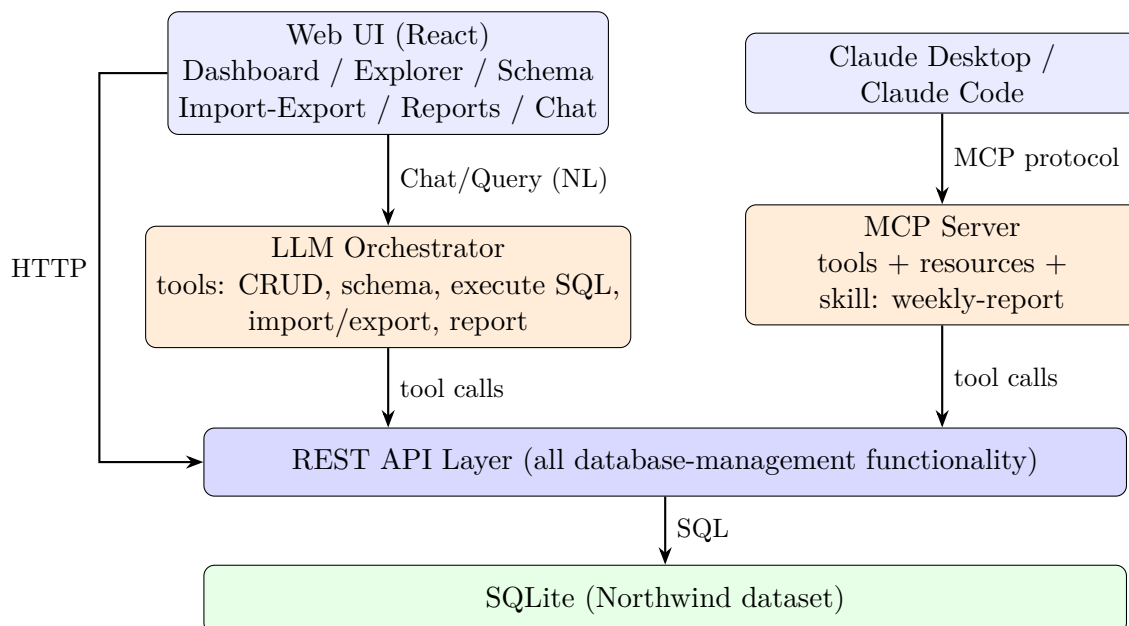


Figure 4: System architecture: the Web UI and the LLM call the same REST API layer; the MCP Server lets Claude Desktop/Code replace the Web UI.

High-level architecture diagram

Why this architecture The system has full database-management functionality that works independently; the LLM is the orchestrating interface to the whole system through tools and MCP, not merely a wrapper layer that answers questions. Users can use Claude Code/Desktop to interact with the app without opening the Web UI — exactly matching the course’s *LLM as main program* criterion.

3.2 Hardware

Written by: 24127139 – Lê Thành Trung

Edited by: 24127139 – Lê Thành Trung

Reviewed by: 24127075 – Nguyễn Sơn Lộc

DataPilot runs entirely on a local machine and requires no dedicated server. Minimum and recommended hardware requirements for the user’s machine:

Hardware	Minimum	Recommended
CPU	Intel Core i3 (dual-core)	Intel Core i5 or higher
RAM	4GB	8GB – 16GB
Storage	5GB free	10GB or more
Internet connection	Wi-Fi or Ethernet	Wi-Fi or Ethernet

An Internet connection is required because the LLM Orchestrator calls the Gemini API (Google) to process natural language. Non-AI features (CRUD, Schema, Import/Export) still work normally while offline.

4 Development Plan

Written by: 22127408 – Kha Vĩnh Thuận

Edited by: 22127408 – Kha Vĩnh Thuận

Reviewed by: 24127139 – Lê Thành Trung

4.1 Requirements analysis

Requirements were gathered through team discussion, brainstorming, discussion and feedback from the supervisor, in order to settle on a feasible scope within 8 weeks. Requirements are split into two groups: functional and non-functional.

Functional requirements

- **Database Explorer:** browse every table in SQLite, search by keyword, perform full CRUD through an interactive table UI, with pagination support.
- **Schema Viewer:** display the full schema of each table (column names, data types, primary/foreign keys, null constraints) in a readable form.
- **Import / Export (CSV):** import data from CSV and export query/table results to CSV; validate the column mapping on import and report errors per row without aborting the whole batch.
- **Chart Dashboard:** display key metrics as bar charts, line charts and summary cards; the chart data is generated by the AI agent when the page loads.

- **Weekly Report Skill (AI):** the LLM automatically chains multiple SQL queries (row counts, new data, aggregations, anomaly detection) to produce a structured weekly report; callable from the Reports page or the MCP server.
- **Chat AI Agent:** a conversational interface that lets users ask natural-language questions; the LLM picks the appropriate tool (`listTables`, `describeTable`, `executeSql...`), executes it, and replies with the raw data.

Non-functional requirements

- **Response time:** every user-facing operation returns a result within 3 seconds under normal conditions; long-running tasks (report generation) have a progress bar and are handled asynchronously.
- **SQL safety (3 layers):**
 - **Layer 1 — System prompt:** the AI agent is prompted to refuse destructive commands (DROP, DELETE, UPDATE) unless the user explicitly confirms.
 - **Layer 2 — Service validation:** the Service layer checks the SQL statement against an allowlist of permitted commands before passing it down to the Repository.
 - **Layer 3 — Parameterized query:** the Repository layer uses only parameterized queries, fully preventing SQL injection.
- **MCP compliance:** the MCP server fully complies with the Model Context Protocol specification so that Claude Desktop/Claude Code can connect without any modification.
- **No authentication:** the tool is used locally by a single person, with no login or authorization, which simplifies the architecture.
- **Maintainability:** the code follows naming conventions, is organized in an N-Layer architecture, and has a clear README so a member can onboard within a day.

Requirements-gathering process

The approach was to brainstorm ideas and discuss with AI: specifically, search for a topic and list a few core features, then use Gemini to suggest features, Perplexity to compare against TablePlus/DBeaver, and Grok for the AI agent design, consolidating everything into a feature list and prioritizing it with MoSCoW. After the supervisor’s feedback, the team dropped multi-database support and added the MCP server as a mandatory deliverable.

Deliverable: Template1-RequirementAnalysis (use cases, user stories).

4.2 Software design

The DataPilot architecture is based on two principles: separation of concerns via an N-Layer architecture, and a single REST API serving both the React UI and the LLM agent, avoiding duplicated logic.

Architecture overview: N-Layer + MVC

- **Presentation Layer:** React 18 SPA (6 pages) + MCP Server (stdio); renders the UI and forwards operations to the backend.
- **Service Layer:** the FastAPI application containing the business logic, the Agent Orchestrator (function-calling loop) and input validation.
- **Repository Layer:** accesses the SQLite data, wrapping every interaction in parameterized queries and type-safe models.

Within the Service Layer, the MVC pattern governs the endpoint structure: the Router is the Controller, the Pydantic model is the Model, and the Jinja2 template (used only for the HTML report preview) is the View. The frontend also follows MVC: a React component is the View, the Zustand store is the Model, and the hook files are the Controller.

Frontend architecture

A React 18 SPA built with Vite (fast HMR) consisting of 6 pages: **Dashboard** (metrics, AI charts), **Database Explorer** (tabular CRUD, search, pagination), **Schema Viewer** (showing columns/types/constraints), **Import/Export** (CSV wizard), **Reports** (view the weekly report, export PDF), **Chat AI Agent** (conversation with a tool-call log). MUI v5 is the component library; Zustand manages client-side state, React Query manages server-state; all HTTP goes through a single Axios instance pointing at FastAPI.

Backend architecture

The FastAPI application follows N-Layer with the `routers/`, `services/`, `repositories/` directories and an `agent/` module containing the Agent Orchestrator. The Orchestrator implements Gemini's function-calling loop: when a message reaches `/api/agent/chat`, it builds a prompt with the tool manifest, sends it to Gemini, checks for tool-call requests, executes them through the Service layer, appends the results to the context, and loops until Gemini's final reply contains no more tool calls. This loop can chain multiple calls — necessary for the weekly report (sometimes 6+ queries).

The tool manifest provided to Gemini includes:

- `listTables()` — list the names of all tables.
- `describeTable(table_name)` — return the columns, types and constraints of a table.
- `executeSql(query)` — run a validated SELECT statement, returning JSON.
- `createRecord(table, data)` — insert a new row.
- `updateRecord(table, id, data)` — update a row by primary key.
- `deleteRecord(table, id)` — delete a row after a safety confirmation.
- `generateWeeklyReport()` — chain multiple analytical queries to produce the weekly report.

- `importCsv(table, file_path)` — import CSV into a table, with column validation.
- `exportCsv(table_or_query)` — export data to CSV.

MCP Server

A standalone Python process implements the MCP server using the MCP Python SDK, running in stdio mode so it is configured directly in `claude_desktop_config.json`. The server exposes all 9 tools as MCP Tools and the table/schema list as MCP Resources, allowing Claude Desktop/Claude Code to use DataPilot as an alternative UI without opening a browser.

Database

SQLite was chosen for its zero-config deployment (no server, account or network configuration required). The application preloads the Northwind dataset (Customers, Orders, Products, Employees...) for immediate demoing and testing.

Deliverable: Template2-Design (detailed architecture, UML, UI design).

4.3 Implementation

Implementation is divided into four sprints, each two weeks long. **Sprint 2 (weeks 3–4) is the highest-risk phase** because of the AI integration (the tool-calling loop) and the SQL safety layer; sprint 4 is reserved for polishing and demo preparation.

Weeks 1–2 — Foundation (Sprint 1)

- Initialize the repo and branches (`main`, `develop`, `feature/*`); set up the venv and manifests (`requirements.txt`, `package.json`).
- Initialize SQLite with Northwind and a migration script.
- Scaffold FastAPI (router/service/repository with placeholder endpoints) and React (Vite, MUI, Router, Zustand, React Query).
- Static UI shell (sidebar, page shells, theme); basic CRUD for Customers/Orders (proof of concept).
- Configure environment variables (`.env` + `python-dotenv`) for the Gemini API key.

Weeks 3–4 — AI core and data layer (Sprint 2 — highest risk)

- Implement the Gemini function-calling loop in the Agent Orchestrator; define the 9 tools and wire them into the Service layer.
- SQL safety validation in the Service layer (allowlist + parameterization); scaffold the MCP server and register the tools.
- Complete the Database Explorer page (browse, search, CRUD) and the Schema Viewer.

- Integration testing: question → tool call → SQL → response, verified end-to-end.
- **Testing gate (end of week 4):** complete the 6 SQL-safety acceptance cases and the AI-core integration tests (end-to-end chat flow); all must pass before moving to sprint 3.

Weeks 5–6 — Feature completion (Sprint 3)

- Chat AI Agent page (message input, streaming display, tool-call log panel).
- Import/Export page (column-mapping preview, CSV download button); Dashboard page (AI charts drawn with Recharts).
- Error handling (React error boundary, structured error responses from FastAPI).
- Complete the MCP server (register Resources, test with Claude Desktop); add the remaining acceptance cases (MCP tools, weekly report) to reach 16 cases and re-run them all.

Weeks 7–8 — Reporting, polishing, demo (Sprint 4)

- Reports page (call the weekly report skill, render markdown with react-markdown); finish `generateWeeklyReport`.
- UI refinements (responsive, loading skeletons, empty states); performance review, add indexes where needed.
- Manual testing of all 6 pages; write and test `run.bat` on a clean Windows machine.
- Write `README.md` for the backend and frontend; prepare and rehearse the demo.

Coding conventions

- `snake_case` for Python functions/variables (per PEP8), `camelCase` for JS/TS functions/variables; `PascalCase` for React components, Python classes and Pydantic models.
- Tool names in the tool manifest (e.g. `listTables`) keep `camelCase` per the function-calling standard, independent of the implementing Python function names.
- Name things using the verb+purpose pattern: Python `fetch_table_schema`, JS/TS `handleDeleteRecord`.
- Commits follow `type(scope): description` with `type` ∈ {`feat`, `fix`, `refactor`, `style`, `docs`, `chore`}; do not mention AI tool names in commits/comments.
- Functions are at most ~40 lines; sensitive values (API keys, paths) live in `.env`, never hard-coded.

Deliverable: source code on GitHub — API, web UI, MCP server, tools, skills.

4.4 Testing

Testing is performed at three levels — unit, integration, manual — together with a set of 16 formal acceptance tests. At the end of week 4 (end of sprint 2) there is a mandatory testing gate: the SQL-safety acceptance cases (6 cases) together with the AI-core integration tests must pass before moving to sprint 3, ensuring the riskiest part is solid before higher-level features are built. The remaining acceptance cases (MCP tools, weekly report) are added and fully passed before the end of sprint 3.

Unit Test

Using pytest (backend) and Vitest (frontend), focusing on: the SQL-safety validator (SELECT passes; INSERT/UPDATE/DELETE/DROP/TRUNCATE are blocked, including keyword case variations, comments and semicolons); the 9 tool functions (valid input, missing parameters, non-existent tables); FastAPI endpoints (via TestClient); Repository functions (run on an in-memory SQLite); and smoke tests of important components (ChatMessage, RecordTable, SchemaGrid).

Integration Test

Checking the interaction between layers on a copy of Northwind: the end-to-end chat flow (a question sent to `/api/agent/chat`, verifying the answer and the correct order of tool calls); the weekly-report chain (call `generateWeeklyReport`, verifying all sections are present and that at least 5 sub-queries run); the Import/Export roundtrip (import then export CSV, byte-for-byte match); MCP tool calls (run the server as a subprocess, connect over stdio, call each tool).

Manual testing

Each member self-tests against ready-made scenarios for the 6 pages: empty table (show an empty state instead of an error), an AI question that cannot be answered (a gentle refusal), large data (import 10,000 rows, Explorer loads within 3 seconds thanks to pagination), a CSV with mismatched columns (report the exact wrong column), and MCP integration (connect Claude Desktop, query end-to-end).

Acceptance Test (16 cases)

The 16 formal test cases are divided into the three most important groups:

- **SQL safety (6 cases):** data-reading statements (SELECT) are allowed to run; data write/update/delete statements (INSERT, UPDATE, DELETE) and table-structure-changing statements (DROP, TRUNCATE) are all blocked; UNION-based SQL injection attacks are also blocked. (*These 6 cases form the testing gate at the end of week 4.*)
- **MCP tools (6 cases):** each main tool has one test case, verifying that the returned result has the expected data format.
- **Weekly report (4 cases):** the generated report runs without errors, has all its sections, has figures matching direct SQL queries, and produces the same result when re-run multiple times (without modifying the data).

Deliverable: Template3-Testing (test plan, test cases, test report).

4.5 Deployment and maintenance

DataPilot runs entirely locally on the user's machine: no cloud infrastructure, no containers, and no network dependencies beyond the HTTPS calls to the Gemini API. This keeps operating costs minimal and allows use in an air-gapped environment if Gemini is later replaced with a local LLM.

Starting the Backend (`run_backend.bat`)

The script automates the entire setup in one action: it checks/creates `venv/`, activates the virtual environment, runs `pip install -r requirements.txt`, copies `.env.example` to `.env` if it does not yet exist (prompting the user to enter the API key), then launches FastAPI via uvicorn at `localhost:8000` with hot-reload.

Starting the Frontend (`run_frontend.bat`)

Similar to the backend: it checks/installs `node_modules/` (`npm install`), runs the Vite dev server at `localhost:5173`, and automatically opens the browser to the application.

MCP Server configuration

Declare an entry in `claude_desktop_config.json` pointing to `mcp_server.py`, running in stdio mode and started automatically by Claude Desktop. No separate script is needed; the `venv` path is written directly in the config to use the correct Python interpreter.

Documentation and maintenance

- `backend/README.md`: setup guide, environment variables, endpoint summary, tool-manifest documentation.
- `frontend/README.md`: setup guide, page descriptions, environment variables, component-library notes.
- `.env.example`: a template of all environment variables with comments; `CHANGELOG.md` updated at the end of each sprint before the review.
- Git history: the commit convention makes it possible to trace every change by feature/fix/maintenance type.

Deliverable: setup/deployment guide, demo video.

5 Human Resources & Costing Plan

Written by: 24127139 – Lê Thành Trung

Edited by: 24127139 – Lê Thành Trung

Reviewed by: 24127075 – Nguyễn Sơn Lộc

Role assignment table:

Role	Assigned to
Project Manager	Kha Vĩnh Thuận
Business Analyst	Nguyễn Sơn Lộc, Lê Thành Trung, Kha Vĩnh Thuận
Frontend Developer	Nguyễn Sơn Lộc, Lê Thành Trung, Kha Vĩnh Thuận
Backend Developer	Nguyễn Sơn Lộc, Lê Thành Trung, Kha Vĩnh Thuận
Tester	Nguyễn Sơn Lộc, Lê Thành Trung, Kha Vĩnh Thuận

Estimated costing table:

Cost item	Amount
Labor (1 month)	~15 million VND
Hosting cost (1 month)	~1 million VND
LLM API cost (1 month)	~500k VND

6 Tools setup

Written by: Whole team

Edited by: 22127408 – Kha Vĩnh Thuận

Reviewed by: 22127408 – Kha Vĩnh Thuận, 24127075 – Nguyễn Sơn Lộc

The team uses the following tools to collaborate and manage the project:

- **Moodle:** receiving the assignment and submitting work per the course rules.
- **Discord:** the team’s main communication channel; technical discussion, progress updates and Jira notifications are shared through the team’s channel.
- **Jira:** managing work by sprint;
each task is assigned to exactly one member, tracking progress per sprint (see Section 1).
- **GitHub:** storing the source code and documents,
with the structure:
 - `/backend` — backend source code (Python, FastAPI).
 - `/frontend` — frontend source code (React, Vite).
 - `/documents` — project documents: proposal, requirements, design, testing.
- **Google Drive:** storing files not suitable for GitHub (large size, binary files, shared copies).
 - `/video` — each member’s screen-recording clips and the final merged version.
 - `/evidence` — screenshots used as evidence (Jira board; AI prompts & history).
 - `/drafts` — draft documents, images and shared resources before they are finalized.

7 AI Usage Declaration

Written by: 22127408 – Kha Vĩnh Thuận

Edited by: 22127408 – Kha Vĩnh Thuận

Reviewed by: 24127075 – Nguyễn Sơn Lộc

7.1 Detailed declaration for each tool

Claude Code & GitHub Copilot

Platform: Claude Code CLI (Anthropic) / GitHub Copilot (GitHub, used in VS Code).

Role: the two main coding-assistant tools, used throughout weeks 1–8.

Produced by AI:

- Boilerplate code for FastAPI routers, Pydantic models and the repository classes.
- React component skeletons (tables, modal dialogs, input fields).
- Initial version of the Gemini function-calling loop based on a high-level description.
- Skeleton code for the MCP server registration (tool and resource definitions).
- Unit-test skeletons for the SQL-safety validator and the API endpoints.
- Debugging suggestions when runtime errors occurred (stack-trace analysis) and suggestions for organizing new modules.

Done by the students:

- Architectural decisions: layer separation, tool-manifest design, MCP resource structure.
- Reviewing and verifying every AI-generated snippet before committing.
- Refining the generated code to match naming conventions and layer boundaries.
- Writing the logic and criteria for the acceptance tests.
- All commits, PR descriptions and merge decisions.

Gemini (Google)

Platform: Google Gemini (gemini.google.com), used as a chat assistant.

Role: report-editing support — grammar checking and sentence rephrasing — weeks 6–8.

Produced by AI:

- Grammar fixes and fluency improvements for report sections drafted by the students.
- Rewriting technically correct but awkward sentences into a more coherent academic style.
- Suggestions for improving the logical flow and connections between paragraphs.
- Summary sentences for complex technical concepts (e.g. describing the MCP protocol).

Done by the students:

- All factual content, technical descriptions and design rationale in the report.
- Choosing which of Gemini’s suggestions to accept, reject or edit.
- The final review to ensure the text accurately reflects the system that was built.
- Structural decisions (section order, heading hierarchy, scope of each part).

Perplexity AI

Platform: Perplexity.ai (an AI assistant with integrated web search).

Role: technology research and comparison — weeks 1–2. *Produced by AI:*

- Comparison of database GUI tools (TablePlus, DBeaver, Beekeeper Studio) to find the gap DataPilot could fill.
- Research summaries on the Model Context Protocol specification and its Python SDK.
- Technology comparisons: FastAPI vs Flask vs Django for the backend; Recharts vs Chart.js vs Victory for the charting library.

Done by the students:

- The final technology decisions and the rationale for each choice.
- Verifying the information Perplexity provided against primary sources (official documentation, GitHub repositories).
- Turning the research results into the project’s concrete requirements and constraints.

Grok (xAI)

Platform: Grok by xAI (grok.com).

Role: idea brainstorming and creativity for the project — weeks 1–2. *Produced by AI:*

- A list of potential features for an LLM-based database-management tool.
- Suggesting the weekly report skill as an appealing illustration of the AI’s multi-step chaining capability.
- Proposing the MCP server as a differentiator versus GUI-only tools.
- Name and slogan suggestions for the project (the name DataPilot was chosen in this session).

Done by the students:

- Evaluating and prioritizing the suggested features according to the team’s capacity.
- The decision to include the MCP server after verifying its feasibility through the Perplexity research.
- All scope decisions are recorded in the requirements document.

ChatGPT (OpenAI)

Platform: ChatGPT (chat.openai.com).

Role: general Q&A and concept clarification — used as needed throughout the project.

Produced by AI:

- Explaining the `asyncio` concept in Python needed for FastAPI background tasks.
- Clarifying SQLite-specific SQL syntax differences from standard SQL.
- Explaining React Query's `staleTime` and `cacheTime` options.
- Example templates of a Zustand store for global state management.

Done by the students:

- The actual implementation code, written by the students after understanding the concept.
- Judging when a concept was understood well enough to apply it independently.
- All debugging beyond concept clarification (reading real stack traces, testing fixes).

Claude (Anthropic)

Platform: Claude.ai (the web interface, Anthropic).

Role: architecture discussion and design planning — weeks 1–3. *Produced by AI:*

- Critiquing the initial N-Layer architecture draft and suggesting improvements.
- Feedback on the tool-manifest design (tool naming, parameter schemas, return-value formats).
- Suggestions for organizing the Agent Orchestrator to clearly separate the loop logic from tool execution.
- Reviewing the MCP server design for its compliance with the protocol specification.

Done by the students:

- The architecture diagrams and the content of the formal design document.
- All final design decisions after weighing Claude's feedback against the team's priorities and constraints.
- Implementing the architecture — here Claude only critiqued, it did not write code.

7.2 Team commitments

The team makes the following formal commitments regarding the use of AI tools in the project:

- **Understand and explain:** every member has read, understood and can explain on their own each part of the report, every submitted line of code and every recorded design decision. AI is used to speed up the work, not to replace understanding.
- **Review and verify:** all AI-generated code is reviewed by at least one member, tested against the acceptance suite, and confirmed correct before committing. No AI-generated code is committed without human review.
- **Accuracy:** every technical claim in the report has been checked against primary sources (official documentation, specifications/RFCs, or direct experimentation). Any AI-assisted text containing errors was corrected before inclusion.
- **Academic integrity:** the AI use described in this declaration is consistent with the course's AI Usage Guidelines. The team does not use AI to avoid the learning objectives; every submitted artifact reflects the team's genuine effort, understanding and judgment.
- **Transparency:** this declaration lists every AI tool used, with nothing omitted or misreported.

By submitting this project, every member confirms that the declaration above is accurate and complete, and takes full responsibility for the correctness and originality of the submitted work.

8 Presentation

Presentation video (≤ 30 minutes), with each member presenting exactly the parts they contributed.

- YouTube link : <https://youtu.be/rFKeLxMiyD4>

9 Reflective Report

24127075 – Nguyễn Sơn Lộc The most useful part for me was the **Preliminary Problem Statement** (and the Proposed Solution part I took part in). Having to clearly state the problem, the operating environment and the constraints *before* writing a single line of code forced the whole team to settle on the scope from the very beginning: choosing local SQLite, using the Gemini API, and adding the MCP server. As a result, all subsequent design decisions and plans rested on a common foundation, avoiding the situation where each person understood the product differently.

Conversely, for a proposal document on its own (Template 0), I found three parts not really necessary. First is the **Presentation (video)**: at the proposal stage there is no product to demo yet, so the video essentially just restates exactly what was written — all of the context, solution, architecture and plan are already fully in the report; the video would be more meaningful at later milestones once there is something runnable to show. Second is **Human Resources & Costing**: the team is a fixed group of three

people with a fixed eight-week timeframe, so personnel can hardly change as in a real company and the role table does not reflect a real decision; the costing table is also just estimated figures (~15 million VND for labor, ~1 million VND for hosting) with no real budget source, no paying party and no agreement between the two sides (customer — developer), so it is more of a formality than a basis for decision-making. Third is the **AI Usage Declaration**: I understand it is a mandatory requirement under the AI Usage Guidelines, but recording and presenting *all* the prompts used is very hard and labor-intensive because the prompts span many weeks, many tools and are largely trial-and-error; in my opinion, an oral session asking directly about the knowledge and design rationale is enough and more effective for checking academic integrity — if a student can explain their own product, they have already proven the most important thing without a large amount of paperwork about prompts.

24127139 – Lê Thành Trung The User Story part may be the most useful, because the main function of the software is to satisfy the needs of the consumer. If a feature has no User Story it is effectively useless, so stating the User Story is an important step to assess the usability of a feature before actually starting to build it.

For example: A home-workout fitness app with an equipment-based workout feature is an example of a feature without a User Story, because there are few or no cases of working out at home with the full equipment available like at a gym.

22127408 – Kha Vĩnh Thuận The most useful part for me was **Proposed Solution — Software Architecture**, because it forced the team to draw the architecture diagram and clearly write the rationale for each component *before* starting to code. In fact, the very process of writing this part helped the team discover a major gap: initially the LLM was designed only as a chatbot that answers questions, not as the central orchestrating component. Thanks to the supervisor’s feedback at the proposal stage, the team added the MCP server and the tool manifest to the design before implementation in time — saving a lot of rework later. A concrete example: without the architecture-diagramming step, the team might have built the Web UI and the LLM Orchestrator with two separate logic layers, leading to code duplication and difficult maintenance; it was the diagram that clarified that both must call a single shared REST API layer.

The less necessary part, in my opinion, is **Hardware Requirements**, because DataPilot is a tool that runs locally on the user’s machine; its hardware requirements are actually very low and do not affect the architecture decision or the development schedule. For example: the entire FastAPI backend, SQLite and the MCP server run smoothly on an old Core i3 laptop with 8GB RAM — far below the “recommended” level in the table. Even so, this part still has value as a reference for the end user during the first installation.